FULL STACK DEVELOPMENT

All Practicals — Step-by-Step Exam Codebook

Aligned to the Nirma University Semester V Full Stack Development practical list (3CS524CC26) and the practical ZIP provided in this chat.

- Practical 1–10 are covered in syllabus order; Git/GitHub deployment is included as Practical 11*.
- Each practical includes setup steps, runnable starter code, key syntax, and common exam variations.
- Use the version specified by the question/handout. Your ZIP contains both Vue 2 and Vue 3 material, so both patterns are shown.
- Use local demo credentials only. Never put production passwords in code or upload them to GitHub.

How to use this in the lab

- Create a separate folder for the practical and save files with the exact names shown.
- For browser-only practicals, open index.html in a browser; for Geolocation, use localhost/HTTPS and allow permission.
- For Node practicals, run node filename.js in the terminal from that folder.
- For database practicals, start the database service and check host, username, password, database, table/collection names.
- Build the minimum working answer first, then add styling or extra features. Test at least one success and one failure case.

Practical 1 — JavaScript in HTML, Node.js Hello World, JSON file

1A. JavaScript embedded in HTML

```
<!-- index.html -->
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>JavaScript Demo</title>
</head>
<body>
  <h1 id="heading">Hello</h1>
  <button onclick="changeText()">Click me</button>
  <script>
    function changeText() {
      document.getElementById("heading").textContent = "Hello from JavaScript!";
    }
  </script>
</body>
</html>
```

Steps: save as index.html, open in browser, click button. JavaScript can be written inline in a script tag or in a separate .js file.

1B. Node.js server-side Hello World

```
/* script.js */
console.log("Hello world");
```

```
# Terminal
node script.js
```

Expected terminal output: Hello world. This is a Node.js script; it prints in the terminal, not in the browser.

1C. Store a JSON object in a file

```
/* app.js */
const fs = require("fs");

const student = {
  rollNo: 101,
  firstName: "Asha",
  email: "asha@example.com"
};

fs.writeFile("student.json", JSON.stringify(student, null, 2), "utf8", (err) => {
  if (err) {
    console.error("Error writing file:", err);
    return;
  }
  console.log("JSON object saved to student.json");
});
```

```
node app.js
```

Remember: JSON.stringify converts a JavaScript object to JSON text. fs.writeFile creates/writes the file. Use the current folder as the working directory.

Practical 2 — JSON objects, external JSON, arrays, to-do list

2A. Create and display a JSON-like JavaScript object

```
/* object.js */
const student = { name: "Riya", age: 20, course: "CSE" };
console.log(student);
console.log(student.name);
console.log(student["course"]);
```

2B. Read an external JSON file

```
/* students.json — valid JSON */
{
  "students": [
    { "id": 1, "name": "Riya" },
    { "id": 2, "name": "Kabir" }
  ]
}
```

```
/* read-json.js */
const fs = require("fs");
fs.readFile("students.json", "utf8", (err, text) => {
  if (err) return console.error(err);
  try {
    const data = JSON.parse(text);
    console.log(data);
    console.log("First student:", data.students[0].name);
  } catch (e) {
    console.error("Invalid JSON:", e.message);
  }
});
```

2C. Multidimensional arrays and element access

```
const marks = [
  ["Riya", 88, 91],
  ["Kabir", 78, 84],
  ["Meera", 95, 93]
];
console.log(marks);       // whole array
console.log(marks[0]);    // first row
console.log(marks[0][0]); // Riya
console.log(marks[1][2]); // 84
```

2D. Complete browser to-do list using a JSON array

```
<!-- todo.html: save and open in browser -->
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>To-do List</title>
<style>
body{font-family:Arial;max-width:600px;margin:30px auto;padding:15px}
input,button{padding:10px;margin:4px}
li{margin:10px 0}.done{text-decoration:line-through;color:gray}
</style>
</head>
<body>
<h1>To-do List</h1>
<input id="taskInput" placeholder="Enter a task">
<button id="addBtn">Add task</button>
<ul id="taskList"></ul>
<script>
let tasks = []; // array of objects: { text, completed }

function renderTasks() {
  const list = document.getElementById("taskList");
  list.innerHTML = "";
  tasks.forEach((task, index) => {
    const li = document.createElement("li");
    const label = document.createElement("span");
    label.textContent = task.text;
    if (task.completed) label.classList.add("done");

    const doneBtn = document.createElement("button");
    doneBtn.textContent = task.completed ? "Undo" : "Complete";
    doneBtn.onclick = () => {
      tasks[index].completed = !tasks[index].completed;
      renderTasks();
    };

    const removeBtn = document.createElement("button");
    removeBtn.textContent = "Remove";
    removeBtn.onclick = () => {
      tasks.splice(index, 1);
      renderTasks();
    };
    li.append(label, doneBtn, removeBtn);
    list.appendChild(li);
  });
}
document.getElementById("addBtn").onclick = () => {
  const input = document.getElementById("taskInput");
  const text = input.value.trim();
  if (!text) return alert("Enter a task first");
```

```
  tasks.push({ text, completed: false });
  input.value = "";
  renderTasks();
};
</script>
</body>
</html>
```

Array methods: push() adds; splice(index, 1) removes; forEach() loops. JSON array here is held in JavaScript memory; reload clears it unless you add Local Storage.

Practical 3 — Responsive website patterns

3A. Viewport, width/max-width, responsive image

```
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Responsive Page</title>
<style>
*{box-sizing:border-box}
body{font-family:Arial;margin:0}
.container{width:90%;max-width:900px;margin:auto;padding:20px}
img{max-width:100%;height:auto;display:block}
</style>
</head>
<body>
<div class="container">
  <h1>Responsive Website</h1>
  <picture>
    <source media="(max-width:600px)" srcset="small.jpg">
    <source media="(min-width:601px)" srcset="large.jpg">
    <img src="large.jpg" alt="Responsive example">
  </picture>
  <p>The content adapts to the screen.</p>
</div>
</body>
</html>
```

Replace small.jpg and large.jpg with files you actually have. If no images are supplied, use one image with img {max-width:100%; height:auto}.

3B. Responsive font using vw

```
<h1 class="responsive-title">Resize the browser</h1>
<style>
.responsive-title {
  font-size: 5vw;
  /* safer bounds */
  font-size: clamp(1.5rem, 5vw, 3.5rem);
}
</style>
```

3C. Media query breakpoint

```
.cards{display:grid;grid-template-columns:repeat(3,1fr);gap:16px}
@media(max-width:768px){
  .cards{grid-template-columns:repeat(2,1fr)}
}
@media(max-width:480px){
  .cards{grid-template-columns:1fr}
}
```

Viewport meta tag makes the page use the device's CSS width. width:90% is flexible; max-width limits the maximum width. vw is a percentage of viewport width. A media query applies CSS only when its condition matches.

Practical 4 — Geolocation, Local Storage, Drag and Drop

4A. Geolocation API

```
<button id="locBtn">Get my location</button>
<p id="location"></p>
<script>
document.getElementById("locBtn").addEventListener("click", () => {
  const output = document.getElementById("location");
  if (!navigator.geolocation) {
    output.textContent = "Geolocation is not supported";
    return;
  }
  navigator.geolocation.getCurrentPosition(
    position => {
      output.textContent =
        "Latitude: " + position.coords.latitude +
        ", Longitude: " + position.coords.longitude;
    },
    error => output.textContent = "Location error: " + error.message
  );
});
</script>
```

Run on localhost or HTTPS and allow browser location permission.

4B. Local Storage

```
<input id="note" placeholder="Write a note">
<button onclick="saveNote()">Save</button>
<button onclick="loadNote()">Load</button>
<p id="result"></p>
<script>
function saveNote() {
  localStorage.setItem("myNote", document.getElementById("note").value);
  alert("Saved");
}
function loadNote() {
  document.getElementById("result").textContent =
    localStorage.getItem("myNote") || "No saved note";
}
</script>
```

For arrays/objects use JSON.stringify(value) when saving and JSON.parse(value) when reading. Do not store passwords or secrets in Local Storage.

4C. Drag and Drop

```
<div id="dragItem" draggable="true" style="padding:15px;background:lightblue;width:140px">
  Drag this item
</div>
<div id="dropZone" style="margin-top:20px;padding:40px;border:2px dashed gray">
  Drop here
</div>
<script>
const item = document.getElementById("dragItem");
const zone = document.getElementById("dropZone");
item.addEventListener("dragstart", event => {
  event.dataTransfer.setData("text/plain", event.target.id);
});
zone.addEventListener("dragover", event => event.preventDefault());
zone.addEventListener("drop", event => {
  event.preventDefault();
  const id = event.dataTransfer.getData("text/plain");
  zone.appendChild(document.getElementById(id));
});
</script>
```

Key events: dragstart, dragover, drop. preventDefault on dragover allows dropping.

Practical 5 — Bootstrap online/offline and student registration

5A. Bootstrap CDN / online

```
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Student Registration</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
<div class="container py-4">
  <h1 class="text-primary mb-4">Student Registration</h1>
  <form>
    <div class="mb-3">
      <label class="form-label" for="roll">Roll Number</label>
      <input class="form-control" id="roll" name="roll" required>
    </div>
    <div class="mb-3">
      <label class="form-label" for="firstName">First Name</label>
      <input class="form-control" id="firstName" name="firstName" required>
    </div>
    <div class="mb-3">
      <label class="form-label" for="mobile">Mobile Number</label>
      <input class="form-control" id="mobile" name="mobile" type="tel" required>
    </div>
    <div class="mb-3">
      <label class="form-label" for="email">Email ID</label>
      <input class="form-control" id="email" name="email" type="email" required>
    </div>
    <div class="mb-3">
      <label class="form-label" for="address">Address</label>
      <textarea class="form-control" id="address" name="address" rows="3"></textarea>
    </div>
    <button class="btn btn-primary" type="submit">Register</button>
  </form>
</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

5B. Bootstrap offline

• Download Bootstrap 5 compiled CSS and bundle JS before the exam (or use the provided lab files).

• Place them in project folders, for example vendor/bootstrap.min.css and vendor/bootstrap.bundle.min.js.

• Replace the CDN links with local paths shown below.

```
<link rel="stylesheet" href="vendor/bootstrap.min.css">
<script src="vendor/bootstrap.bundle.min.js"></script>
```

5C. Bootstrap classes to remember

container/container-fluid, row, col-12, col-md-6, g-3, p-3, m-2, mt-3, text-center, text-primary, bg-dark, navbar, card, btn btn-primary, form-control, form-label, alert alert-success. Bootstrap's grid is based on 12 columns; col-12 col-md-6 means full width on small screens and half width on medium screens and above.

Practical 6 — Tailwind CSS web page

CDN demo page with a registration form

```
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Tailwind Registration</title>
<script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="min-h-screen bg-slate-100 p-4">
<main class="mx-auto max-w-xl rounded-xl bg-white p-6 shadow">
  <h1 class="mb-5 text-2xl font-bold text-blue-700">Student Registration</h1>
  <form class="space-y-3">
    <input class="w-full rounded border p-2" placeholder="Roll number" required>
    <input class="w-full rounded border p-2" placeholder="First name" required>
    <input class="w-full rounded border p-2" type="tel" placeholder="Mobile number" required>
    <input class="w-full rounded border p-2" type="email" placeholder="Email ID" required>
    <textarea class="w-full rounded border p-2" placeholder="Address"></textarea>
    <button class="rounded bg-blue-600 px-4 py-2 font-semibold text-white hover:bg-blue-700">
      Register
    </button>
  </form>
</main>
</body>
</html>
```

Tailwind class translation

Purpose	Example class
Spacing	p-4, px-4, py-2, m-4, mt-2, gap-4
Width/layout	w-full, max-w-xl, mx-auto, flex, grid
Text	text-xl, text-center, font-bold, text-blue-700
Background/border	bg-white, bg-slate-100, border, rounded-lg
Effects	shadow, shadow-md, hover:bg-blue-700
Responsive	md:grid-cols-2, sm:p-6

Tailwind is utility-first: use multiple small classes to style one element. The CDN approach requires internet access. If the exam expects a local setup, use the project's installed Tailwind build instead.

Practical 7 — Vue.js custom directives (Vue 3 and Vue 2)

Your practical list asks for three items: transform text to uppercase when clicked, create a dynamic list, and format dates. Use the Vue version specified by your faculty/handout.

7A. Vue 3 — all three features

```
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Vue Directives</title>
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
<div id="app">
  <h2 v-uppercase>click this sentence to uppercase</h2>

  <h3>Dynamic list</h3>
  <input v-model="newItem" @keyup.enter="addItem" placeholder="New item">
  <button @click="addItem">Add</button>
  <ul><li v-for="(item, index) in items" :key="index">{{ item }}</li></ul>

  <h3>Date formatting</h3>
  <p v-date="'2026-10-25'"></p>
</div>
<script>
const app = Vue.createApp({
  data() {
    return { newItem: "", items: ["HTML", "CSS", "JavaScript"] };
  },
  methods: {
    addItem() {
      const value = this.newItem.trim();
      if (value) this.items.push(value);
      this.newItem = "";
    }
  }
});

app.directive("uppercase", {
  mounted(el) {
    el.style.cursor = "pointer";
    el.addEventListener("click", () => {
      el.textContent = el.textContent.toUpperCase();
    });
  }
});

app.directive("date", {
  mounted(el, binding) {
    const date = new Date(binding.value);
    el.textContent = Number.isNaN(date.getTime())
      ? "Invalid date"
      : date.toLocaleDateString(undefined, {
          year: "numeric", month: "long", day: "numeric"
        });
  }
});
app.mount("#app");
</script>
</body>
</html>
```

7B. Vue 2 — uppercase custom directive pattern

```
<!-- Load Vue 2 script before this code -->
<div id="app">
  <p v-uppercase>click me to uppercase</p>
</div>
<script>
Vue.directive("uppercase", {
  inserted: function (el) {
    el.style.cursor = "pointer";
    el.addEventListener("click", function () {
      el.textContent = el.textContent.toUpperCase();
    });
  }
});
new Vue({ el: "#app" });
</script>
```

Vue 2 uses new Vue({el:'#app'}), Vue.directive(), and hooks such as inserted. Vue 3 uses Vue.createApp(), app.directive(), and hooks such as mounted. Directives are named with v- in HTML; register them without the v- prefix in JavaScript.

Practical 8 — Node.js + MySQL create, insert, update

Step 1: SQL setup (run in MySQL/phpMyAdmin)

```
CREATE DATABASE IF NOT EXISTS college;
USE college;
CREATE TABLE IF NOT EXISTS students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150)
);
INSERT INTO students (name, email)
VALUES ('Asha', 'asha@example.com');
SELECT * FROM students;
```

Step 2: Install the driver

```
npm init -y
npm install mysql
```

Step 3: connect and perform operations

```
/* mysql-practical.js */
const mysql = require("mysql");
const connection = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "", // replace with your local MySQL password
  database: "college"
});

connection.connect(err => {
  if (err) {
    console.error("Connection error:", err.message);
    return;
  }
  console.log("Connected to MySQL");

  connection.query(
    "INSERT INTO students (name, email) VALUES (?, ?)",
    ["Riya", "riya@example.com"],
    (err, result) => {
      if (err) console.error(err.message);
      else {
        console.log("Inserted ID:", result.insertId);
        connection.query(
          "UPDATE students SET name = ? WHERE id = ?",
          ["Riya Patel", result.insertId],
          (err2, result2) => {
            if (err2) console.error(err2.message);
            else console.log("Updated rows:", result2.affectedRows);
            connection.end();
          }
        );
      }
    }
  );
});
```

Run: node mysql-practical.js. Start MySQL first. Use the actual credentials configured on your lab machine. The ? placeholders parameterize values.

8B. MySQL CRUD reference

```
INSERT INTO students (name, email) VALUES ('Meera', 'meera@example.com');
SELECT * FROM students;
SELECT * FROM students WHERE id = 1;
UPDATE students SET email = 'new@example.com' WHERE id = 1;
DELETE FROM students WHERE id = 1;
SELECT DISTINCT name FROM students;
DROP TABLE IF EXISTS students;
```

Do not run UPDATE or DELETE without a WHERE clause unless you intentionally want to change/remove all rows. DROP TABLE removes the table structure and its data.

Practical 9 — Role-based authentication and access control

Authentication = verify identity. Authorization = check permission. The syllabus says JavaScript; the practical ZIP includes a Node HTTP-style server example. This runnable demo uses in-memory demo accounts only.

server.js — role-based access

```
const http = require("http");
const users = [
  { username: "admin", password: "1234", role: "admin" },
  { username: "student", password: "1234", role: "user" }
];

http.createServer((req, res) => {
  res.setHeader("Content-Type", "application/json");

  // Demo only: credentials passed in headers for lab illustration.
  const username = req.headers.username;
  const password = req.headers.password;
  const user = users.find(u =>
    u.username === username && u.password === password
  );

  if (!user) {
    res.writeHead(401);
    return res.end(JSON.stringify({ message: "Authentication failed" }));
  }

  if (req.url === "/admin" && user.role !== "admin") {
    res.writeHead(403);
    return res.end(JSON.stringify({ message: "Admin access only" }));
  }

  res.writeHead(200);
  res.end(JSON.stringify({
    message: "Access granted",
    username: user.username,
    role: user.role
  }));
}).listen(3000, () => console.log("Server at http://localhost:3000"));
```

Status codes to remember

200 = OK; 201 = Created; 400 = Bad Request; 401 = credentials missing/invalid; 403 = authenticated but not permitted; 404 = route not found; 500 = server error.

Security note: these credentials are for a classroom demonstration only. A real app must use HTTPS, hashed passwords, sessions/tokens, server-side validation, and secrets stored outside source code.

Practical 10 — Node.js + MongoDB CRUD

Step 1: start MongoDB and install driver

```
npm init -y
npm install mongodb
```

Step 2: one script demonstrating Create, Read, Update, Delete

```
/* mongo-practical.js */
const { MongoClient } = require("mongodb");
const uri = "mongodb://127.0.0.1:27017";

async function main() {
  const client = new MongoClient(uri);
  try {
    await client.connect();
    console.log("Connected to MongoDB");

    const db = client.db("college");
    const students = db.collection("students");

    // CREATE
    const insertResult = await students.insertOne({
      name: "Asha", age: 20, course: "CSE"
    });
    console.log("Created ID:", insertResult.insertedId);

    // READ
    const allStudents = await students.find({}).toArray();
    console.log("All students:", allStudents);

    // UPDATE
    const updateResult = await students.updateOne(
      { _id: insertResult.insertedId },
      { $set: { age: 21 } }
    );
    console.log("Modified:", updateResult.modifiedCount);

    // DELETE
    const deleteResult = await students.deleteOne({
      _id: insertResult.insertedId
    });
    console.log("Deleted:", deleteResult.deletedCount);
  } catch (error) {
    console.error("MongoDB error:", error.message);
  } finally {
    await client.close();
  }
}
main();
```

Run: node mongo-practical.js. The MongoDB service must be running. This code deletes the document it just created, so it is repeatable.

MongoDB query patterns

```
db.students.insertOne({ name: "Asha", age: 20 });
db.students.find({});
db.students.find({ name: "Asha" });
db.students.updateOne({ name: "Asha" }, { $set: { age: 21 } });
db.students.deleteOne({ name: "Asha" });
```

SQL uses tables and rows. MongoDB uses collections and documents. MongoDB documents have an _id field. In Node, find() returns a cursor, so use toArray() to print results.

Practical 11* — Git, GitHub and deployment basics

Git commands to remember

```
git --version
git init
git status
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/USERNAME/REPOSITORY.git
git push -u origin main
```

Create an empty GitHub repository first, replace the URL with your own repository URL, and never commit .env files, passwords, database credentials, or API keys.

Useful .gitignore

```
node_modules/
.env
.DS_Store
```

Generic deployment checklist

• Ensure package.json has the correct dependencies and start script.

• Use process.env.PORT when the hosting provider supplies a port.

• Move credentials to environment variables.

• Push the code to GitHub and connect the repository to the chosen hosting provider.

• Configure database network access/credentials for the deployed app; localhost database URLs will not work from most hosted servers.

```
/* Example package.json script fragment */
{
  "scripts": {
    "start": "node server.js"
  }
}
```

Reusable full-stack form → backend → database pattern

If the examiner asks for a small website, use this sequence: HTML form → JavaScript submit handler → HTTP request → Node backend route → validate → database insert → JSON response → show result.

Frontend submit pattern

```
const form = document.querySelector("#studentForm");
form.addEventListener("submit", async event => {
  event.preventDefault();
  const data = Object.fromEntries(new FormData(form).entries());

  const response = await fetch("/api/students", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data)
  });
  const result = await response.json();
  document.querySelector("#message").textContent = result.message;
});
```

Express backend skeleton (if Express is allowed)

```
npm install express
```

```
const express = require("express");
const app = express();
app.use(express.json());
app.use(express.static("public"));

app.get("/api/students", (req, res) => {
  res.json({ message: "Student list endpoint" });
});

app.post("/api/students", (req, res) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return res.status(400).json({ message: "Name and email required" });
  }
  // Replace this line with a SQL INSERT or MongoDB insertOne.
  res.status(201).json({ message: "Student received", student: { name, email } });
});

app.listen(process.env.PORT || 3000, () => console.log("Server started"));
```

Express is an optional general-purpose pattern; the listed database practicals specifically ask for Node.js with MySQL/MongoDB. If the question or lab template requires built-in http, use the Practical 9 HTTP pattern instead.

Final exam checklist and viva quick answers

Common errors

• Cannot find module: run npm install for the named package in the correct folder.

• ECONNREFUSED: database/server is not running or host/port is wrong.

• Access denied: verify database username/password and permissions.

• Table/collection not found: check spelling and create it first.

• HTML/JS not working: check script src path, element id spelling, and browser console.

• Geolocation denied: allow permission and use localhost/HTTPS.

• Vue directive not working: check Vue 2 vs Vue 3 and script loading order.

• Bootstrap/Tailwind styles missing: check internet/CDN or local file paths.

Viva questions

Q. What is full stack development? Frontend, backend and database development together.

Q. HTML/CSS/JavaScript? Structure, presentation, behaviour.

Q. What is JSON.stringify? Converts a JS value into JSON text.

Q. What is JSON.parse? Converts valid JSON text into a JS value.

Q. What is Local Storage? Browser key/value storage that persists across reloads.

Q. What is a media query? Conditional CSS for a viewport/device condition.

Q. Bootstrap vs Tailwind? Prebuilt component-oriented CSS framework vs utility-first CSS framework.

Q. What is a Vue directive? A v- prefixed instruction used for DOM behaviour or binding.

Q. What is Node.js? JavaScript runtime outside the browser.

Q. What is CRUD? Create, Read, Update, Delete.

Q. What is a primary key? A column or set of columns uniquely identifying a row.

Q. SQL vs MongoDB? Relational tables versus document collections.

Q. Authentication vs authorization? Verify identity versus check permissions.

Q. Why use parameterized SQL? Safer handling of input and protection against SQL injection.

Q. Why close database clients? Release connections/resources after operations.

Best last-minute strategy: practise one form, one responsive CSS layout, one Node server, one MySQL insert/select/update, one MongoDB CRUD script, and the three Vue directive tasks. Those are reusable patterns, not one-off answers.